

Rapport de Projet : Jeu de Taquin

Table des matières

1. Introduction
2. Description des fonctionnalités
3. Structure du programme et architecture
4. Représentation des données
5. Algorithme de mélange et preuve de résolubilité
6. Conclusion personnelle

1. Introduction

Ce projet consiste à la conception et le développement d'un jeu de Taquin en C, utilisant la bibliothèque graphique de l'IUT.

Le Taquin est un jeu seul dont l'objectif est de reconstituer une image séparée en morceaux en faisant glisser les parties sur une case vide retirée en préalable.

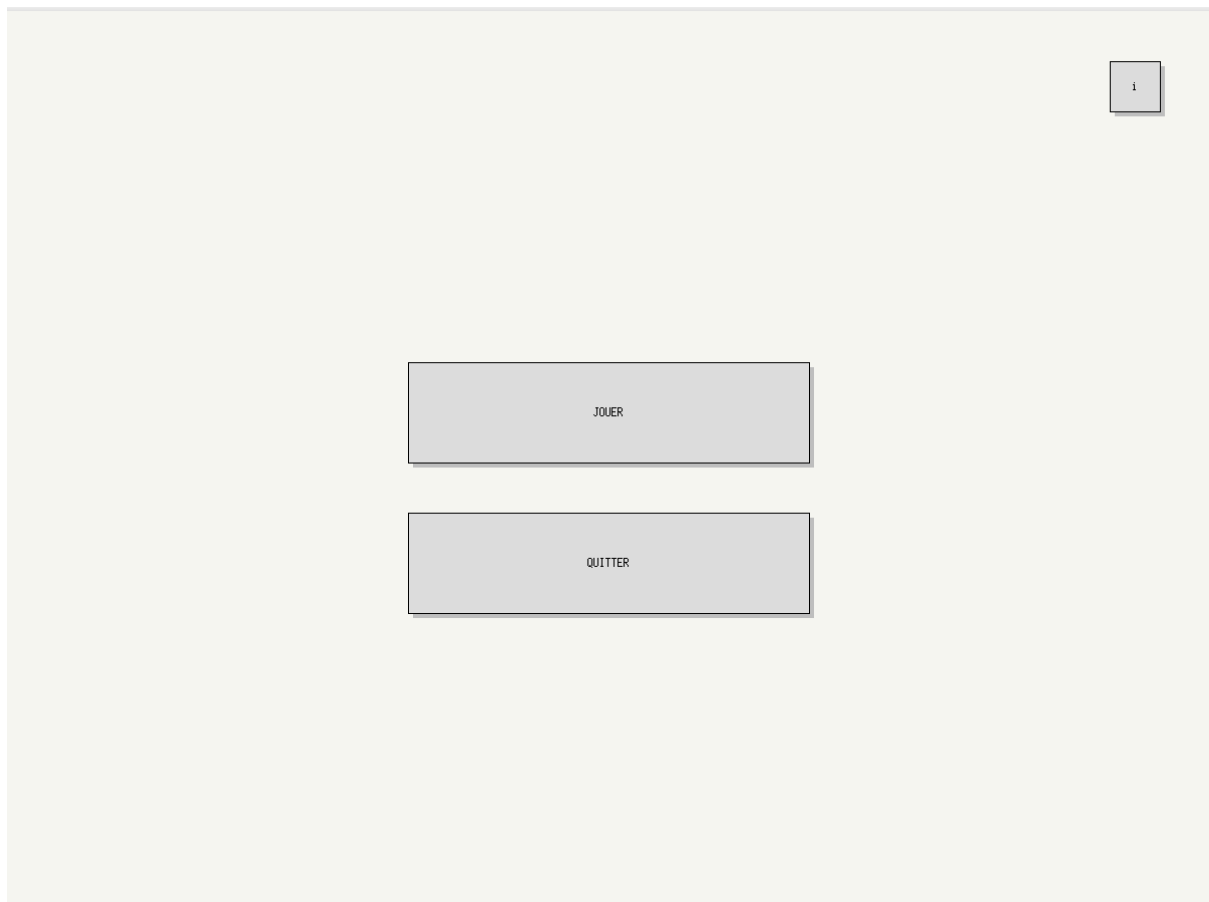
L'objectif de ce développement était double : d'une part, maîtriser la gestion de la mémoire dynamique et des structures de données en C pour représenter l'état du jeu, et d'autre part, concevoir une architecture logicielle modulaire séparant la logique du jeu de son interface graphique. Le programme final permet au joueur de choisir une image, de paramétrer la difficulté (taille de la grille) et de résoudre le puzzle grâce à une interaction fluide à la souris ou au clavier.

2. Description des fonctionnalités

Le programme a été pensé pour offrir une expérience utilisateur intuitive et configurable. Voici les principales fonctionnalités implémentées :

2.1. Menu Principal et Navigation

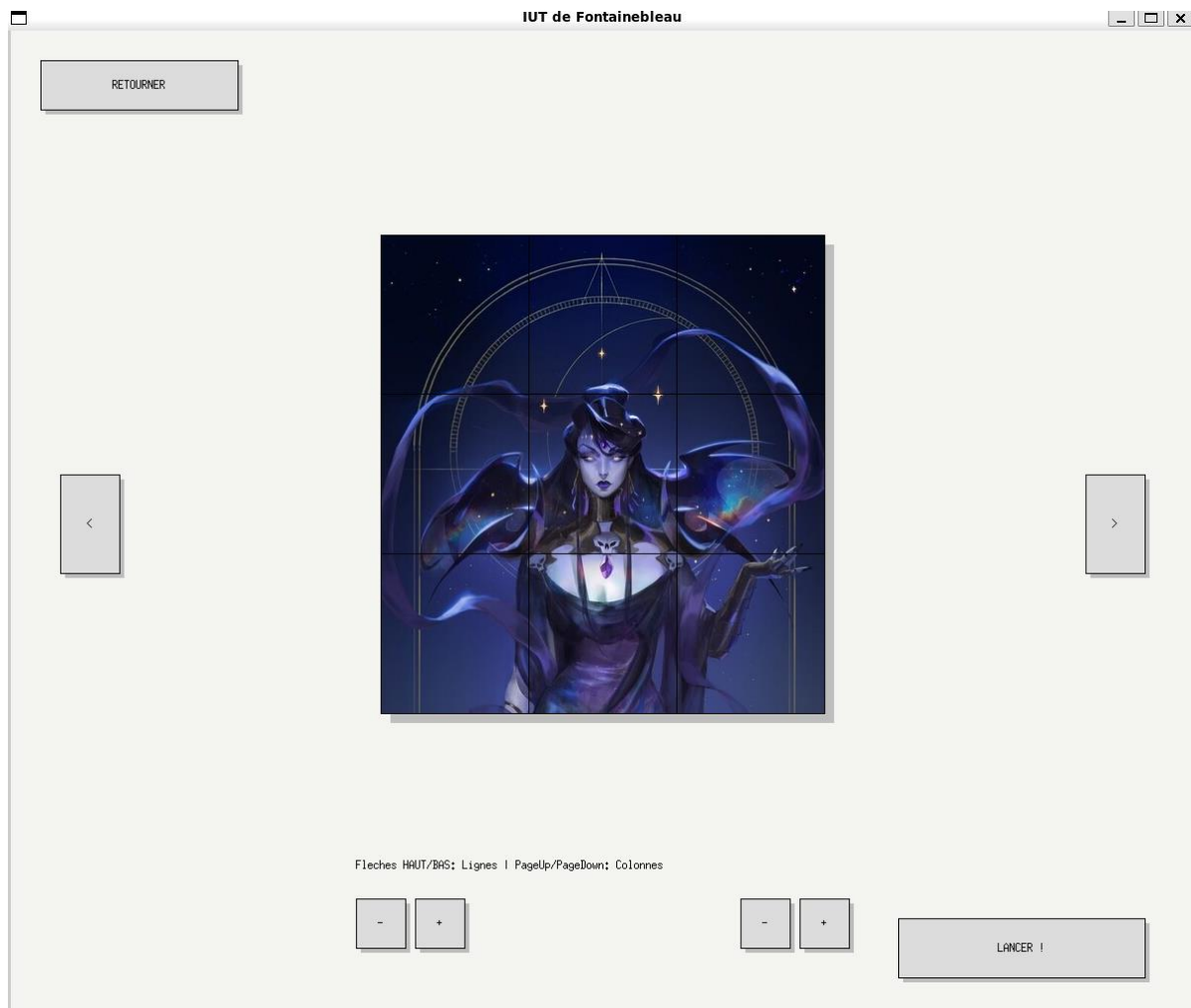
Dès le lancement, l'utilisateur accède à un menu principal permettant de lancer une partie, de consulter l'aide ou de quitter. La navigation entre les différents écrans (Menu, Choix, Jeu, Aide) est gérée par une machine à états, assurant des transitions fluides.



2.2. Configuration de la Partie

Avant de jouer, un écran de paramétrage permet de :

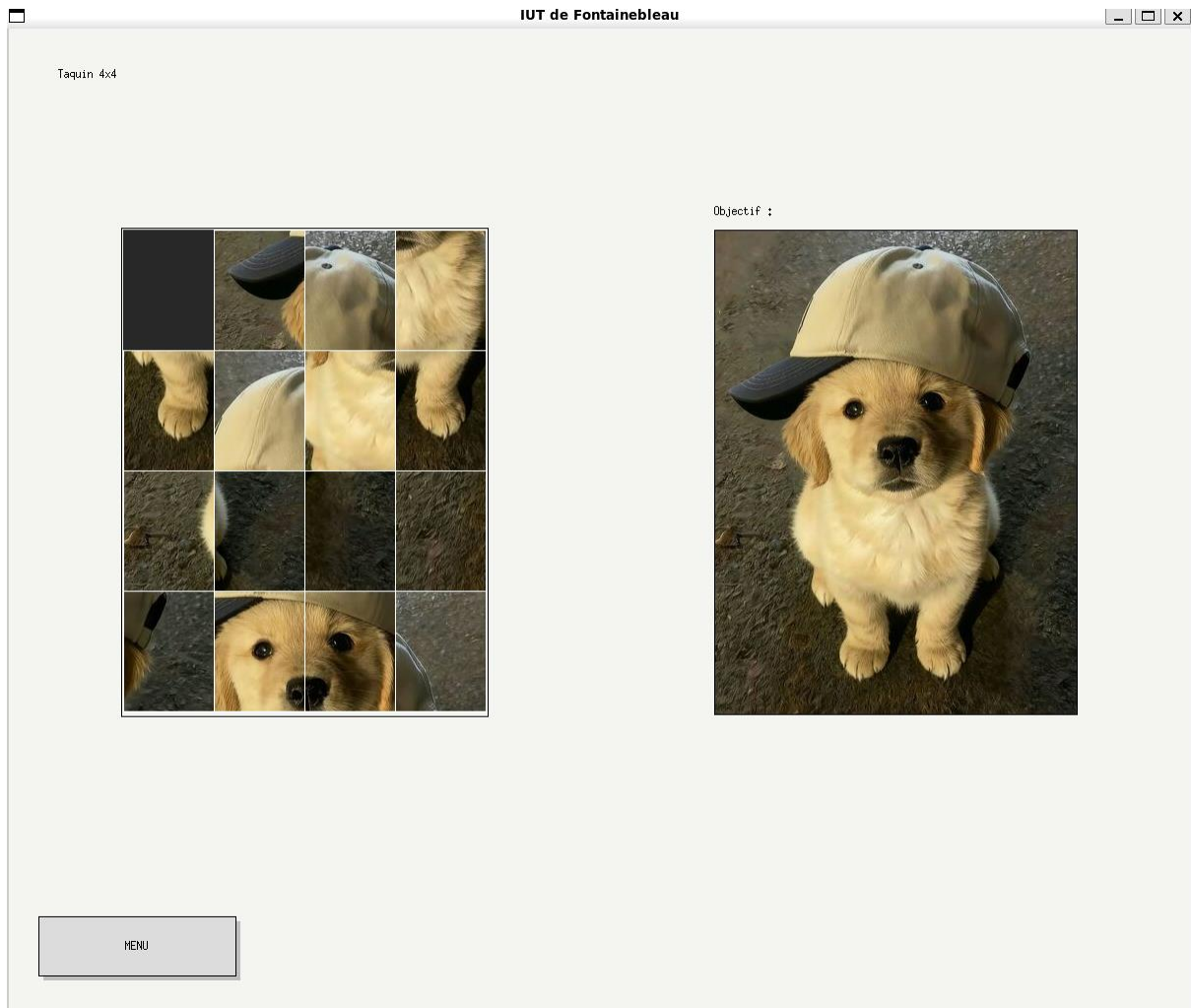
- **Choisir l'image :** Navigation parmi une liste d'images prédéfinies (Zelda, paysages, etc.) via un carrousel.
- **Modifier la taille de la grille :** L'utilisateur peut augmenter ou diminuer le nombre de lignes et de colonnes (de 3x3 à 8x8) dynamiquement. Le programme recalcule instantanément le découpage de l'image.



2.3. Interface de Jeu (In-Game)

Une fois la partie lancée, l'écran de jeu affiche :

- La grille de Taquin mélangée à gauche.
- L'aperçu de l'image finale (objectif) à droite pour aider le joueur.
- **Contrôles** : Le déplacement des cases se fait soit par un clic de souris sur une case adjacente au vide, soit via les flèches directionnelles du clavier.
- **Détection de victoire** : Lorsqu'une pièce est déplacée, le programme vérifie si l'ordre des images est correct. Un message de victoire apparaît alors.



3. Structure du programme

Pour garantir la maintenabilité et la clarté du code, nous avons opté pour un découpage modulaire.

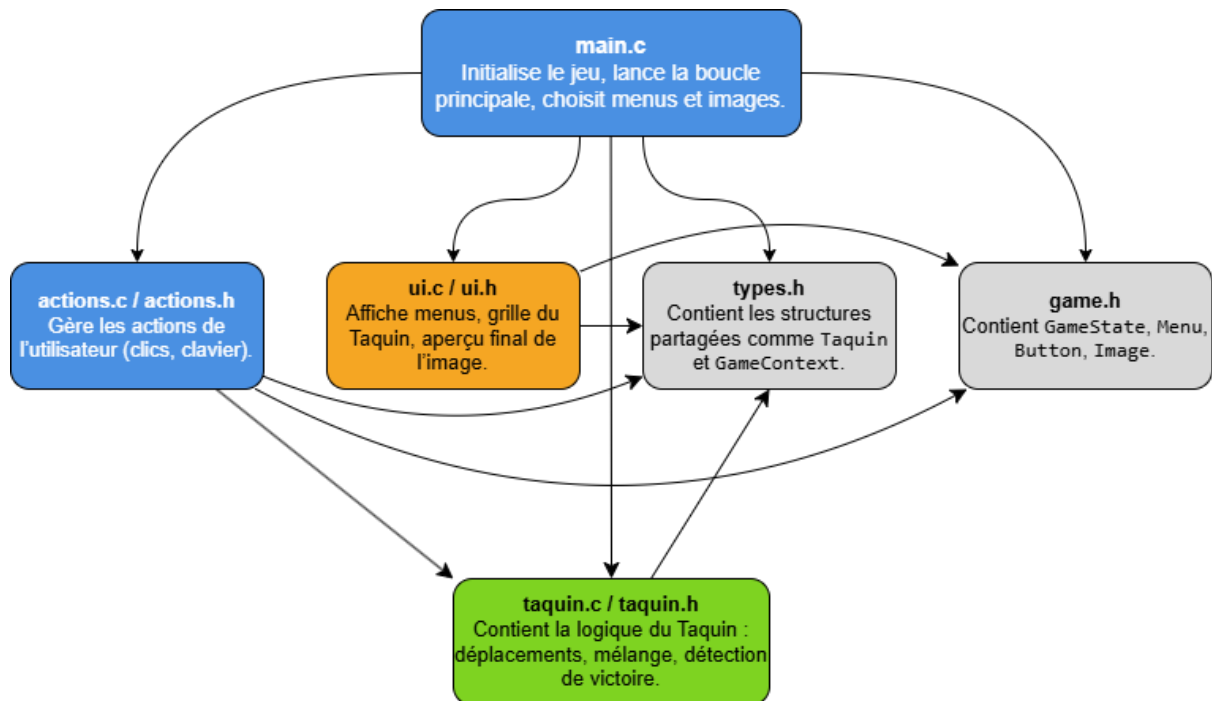
3.1. Justification du découpage

Le projet est réparti en plusieurs fichiers sources, chacun ayant une responsabilité unique :

- **Le Modèle (`taquin.c`, `taquin.h`)** : Contient la logique pure du jeu. Il gère la grille de données, les règles de déplacement et la vérification de victoire. Il est indépendant et ne sait pas qu'une interface graphique existe.
- **La Vue (`ui.c`, `ui.h`)** : Gère tout ce qui est affiché à l'écran (dessin des boutons, découpage des images, rendu de la grille). Elle dépend des données du modèle mais ne les modifie pas directement.
- **Le Contrôleur (`action.c`, `action.h`, `main.c`)** : Fait le lien entre les entrées utilisateur (clic, clavier) et les modifications du modèle.

- **Les Types partagés (types.h) :** Définit les structures communes (GameContext, Taquin) pour éviter les dépendances de cercle entre les fichiers.

3.2. Diagramme d'architecture



4. Explication des données

L'état d'une partie est enregistré dans deux structures principales définies dans **types.h**.

4.1. La structure Taquin (Le plateau)

C'est la structure centrale des données. Elle contient :

- **int **grid :** Un tableau dynamique à deux dimensions (allocation pointeur de pointeurs). Chaque case contient un entier représentant l'identifiant original de la portion d'image.
 - **Type :** Entier (int).
 - **Valeur :** 0 représente la case vide (le "trou"). Les valeurs 1 à N-1 représentent les tuiles d'image.
- **int row, col :** Les dimensions de la grille.
- **int empty_cell[2] :** Un tableau stockant les coordonnées [ligne, colonne] actuelles de la case vide pour optimiser la recherche lors des mouvements.

4.2. La structure GameContext (L'application)

Cette structure agit comme un conteneur global (passé en pointeur à toutes les fonctions) pour éviter l'usage de variables globales. Elle contient :

- L'instance de la structure Taquin.
- L'état de l'interface (menu actuel, index de l'image choisie).
- Les valeurs de contrôle (run pour la boucle principale, redraw pour rafraîchir l'affichage, win pour l'état de victoire).

5. Analyse de l'algorithme de mélange

L'un des défis majeurs de ce projet Taquin était de garantir que la disposition initiale soit résoluble. Si l'on plaçait les tuiles totalement au hasard, 50% des configurations seraient impossibles à résoudre (à cause du problème de parité des permutations).

5.1. Fonctionnement de l'algorithme

Notre algorithme, situé dans la fonction `taquin_shuffle`, procède comme suit :

1. On part de la **grille résolue** (état final).
2. On effectue une "Marche Aléatoire" (Random Walk) : on simule un grand nombre de mouvements valides choisis au hasard.
 - a. La variable `force_melange` détermine le nombre d'itérations (calculée par $(\text{lignes} * \text{colonnes} * 100)$).
3. Pour chaque itération, on choisit une direction aléatoire (Haut, Bas, Gauche, Droite). Si le mouvement est légal (ne sort pas de la grille), on échange la case vide avec la case adjacente.
4. À la fin du mélange, on remonte artificiellement la case vide en haut à gauche (position (0,0)) par des coups valides pour la retirer et standardiser la partie.

5.2. Analyse de l'algorithme de mélange et preuve de résolubilité

L'un des défis majeurs de la programmation d'un Taquin est de garantir que la disposition initiale des tuiles est résoluble. En effet, si l'on plaçait les tuiles de manière totalement aléatoire dans la grille, 50 % des configurations obtenues seraient mathématiquement impossibles à résoudre (problème lié à la parité des permutations).

Pour pallier ce problème, notre algorithme implémenté dans la fonction `"taquin_shuffle"` n'effectue pas un placement aléatoire absolu, mais procède par une "Marche Aléatoire" (Random Walk = mélanges par pas) à partir de la solution.

Fonctionnement de l'algorithme :

1. Le programme initialise la grille dans son état résolu (état final).
2. Il effectue une boucle de mélange dont le nombre d'itérations est proportionnel à la taille de la grille (Nombre de cases x 100).
3. A chaque itération, une direction est choisie au hasard (Haut, Bas, Gauche ou Droite).
4. Si le mouvement est légal (c'est-à-dire que la case vide ne sort pas de la grille), l'échange est effectué.
5. A la fin du mélange, l'algorithme remonte la case vide vers la position (0,0) par des mouvements légaux pour standardiser le début de partie.

Démonstration de la résolubilité :

Théorème : Tout état obtenu en appliquant une suite finie de mouvements légaux à partir de l'état résolu est nécessairement résoluble.

Démonstration : Soit S_{final} l'état résolu du Taquin (l'objectif). Soit M un mouvement légal (déplacer une tuile vers la case vide). Une propriété fondamentale du Taquin est la réversibilité : pour tout mouvement M transformant un état A en état B , il existe un mouvement inverse M' permettant de revenir de B vers A .

Notre algorithme de mélange génère l'état initial du jeu (S_{initial}) en appliquant une suite de k mouvements aléatoires ($m_1, m_2, \dots, m_k$) à partir de l'état résolu S_0 .

Puisque chaque mouvement est inversible, il existe mathématiquement une suite de mouvements (l'inverse de la séquence de mélange) qui permet de faire le chemin inverse de S_{initial} vers S_{final} .

Conclusion : En simulant le mélange "à l'envers" (en partant de la fin et en jouant des coups valides), nous garantissons que la configuration générée est inversible et donc résoluble. La disposition obtenue permet donc toujours au joueur de réussir.

6. Conclusion personnelle

[Djibril Guinet]

J'ai bien aimé ce projet car il a été une excellente opportunité de mettre en pratique concrète ce qu'on apprend en C depuis le début de l'année. Les difficultés principales selon moi résidaient dans le fait d'apprendre à bien séparer les fichiers ainsi que de créer un Makefile. Ayant l'habitude de faire des jeux utilisant les interfaces graphiques et l'orienté objet (même notre programme en C diffère de la POO que je connais), j'ai

trouvé la bibliothèque graphique intuitive et intéressante, et je me suis amusé avec les formules mathématiques en tout genre nécessaires durant ce projet.

[Anthony Millerioux]

Ce projet a été simple à démarrer mais assez difficile à finaliser. La partie la plus complexe a été maîtriser et optimiser la bibliothèque graphique pour que le jeu reste fluide. Au début, nous avons tendance à tout mettre dans peu de fichiers, mais le découpage en modules (UI, Actions, Modèle, Types) s'est avéré plus clair et plus facile à gérer que prévu. La manipulation des images pour découper dynamiquement la grille fut un défi technique enrichissant, qui nous a permis de mieux comprendre la gestion de la mémoire et des structures en C. Ce projet nous a donné une bonne expérience en architecture modulaire, combinant logique de jeu, interface graphique et état global.